

ID	b_1	b_2	b_3
G1	13	17	5
G2	7	13	3
G3	9	17	6
G4	6	13	5
G5	9	21	2
G6	17	15	5
G7	3	13	7
G8	5	13	6
G9	12	21	5

(H) MWC with Base $b = 2^{16}$

state: x, y (unsigned 32-bit)
 initialize: $1 \leq x, y \leq 2^{16} - 1$
 update: $x \leftarrow a (x \ \& \ [2^{16} - 1]) + (x \gg 16)$
 $y \leftarrow b (y \ \& \ [2^{16} - 1]) + (y \gg 16)$
 can use as random: $(x \ll 16) + y$
 can use in bit mix: same, or (with caution) x or y
 can improve by: output 32-bit xorshift successor
 period: $(2^{16}a - 2)(2^{16}b - 2)/4$ (product of two primes)

ID	a	b
H1	62904	41874
H2	64545	34653
H3	34653	64545
H4	57780	55809
H5	48393	57225
H6	63273	33378

(I) LCG Modulo 2^{32}

state: x (unsigned 32-bit)
 initialize: any value
 update: $x \leftarrow ax + c \pmod{2^{32}}$
 can use as random: not recommended
 can use in bit mix: not recommended
 can improve by: output 32-bit xorshift successor
 period: 2^{32}

ID	a	c (any odd ok)
I1	1372383749	1289706101
I2	2891336453	1640531513
I3	2024337845	797082193
I4	32310901	626627237
I5	29943829	1013904223

(J) MLCG Modulo 2^{32}

state:	x (unsigned 32-bit)
initialize:	$x \neq 0$
update:	$x \leftarrow ax \pmod{2^{32}}$
can use as random:	not recommended
can use in bit mix:	not recommended
can improve by:	output 32-bit xorshift successor
period:	2^{30}

ID	a
J1	1597334677
J2	741103597
J3	1914874293
J4	990303917
J5	747796405

A high-quality, if somewhat slow, combined generator is

$$\text{Ranlim32} \equiv [\text{G3}_l(\text{I2}) + \text{G1}_r] \wedge [\text{G6}_l(\text{H6}_b) + \text{H5}_b] \quad (7.1.6)$$

implemented as

```
struct Ranlim32 {
    Uint u,v,w1,w2;
    Ranlim32(Uint j) : v(2244614371U), w1(521288629U), w2(362436069U) {
        u = j ^ v; int32();
        v = u; int32();
    }
    inline Uint int32() {
        u = u * 2891336453U + 1640531513U;
        v ^= v >> 13; v ^= v << 17; v ^= v >> 5;
        w1 = 33378 * (w1 & 0xffff) + (w1 >> 16);
        w2 = 57225 * (w2 & 0xffff) + (w2 >> 16);
        Uint x = u ^ (u << 9); x ^= x >> 17; x ^= x << 6;
        Uint y = w1 ^ (w1 << 17); y ^= y >> 15; y ^= y << 5;
        return (x + v) ^ (y + w2);
    }
    inline Doub doub() { return 2.32830643653869629E-10 * int32(); }
    inline Doub truedoub() {
        return 2.32830643653869629E-10 * ( int32() +
            2.32830643653869629E-10 * int32() );
    }
};
```

ran.h

Note that the `doub()` method returns floating-point numbers with only 32 bits of precision. For full precision, use the slower `truedoub()` method.

CITED REFERENCES AND FURTHER READING:

- Gentle, J.E. 2003, *Random Number Generation and Monte Carlo Methods*, 2nd ed. (New York: Springer), Chapter 1.
- Marsaglia, G 1968, "Random Numbers Fall Mainly in the Planes", *Proceedings of the National Academy of Sciences*, vol. 61, pp. 25–28.[1]

- Entacher, K. 1997, "A Collection of Selected Pseudorandom Number Generators with Linear Structures", Technical Report No. 97, Austrian Center for Parallel Computation, University of Vienna. [Available on the Web at multiple sites.][2]
- Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley).[3]
- Schrage, L. 1979, "A More Portable Fortran Random Number Generator," *ACM Transactions on Mathematical Software*, vol. 5, pp. 132–138.[4]
- Park, S.K., and Miller, K.W. 1988, "Random Number Generators: Good Ones Are Hard to Find," *Communications of the ACM*, vol. 31, pp. 1192–1201.[5]
- L'Ecuyer, P. 1997 "Uniform Random Number Generators: A Review," *Proceedings of the 1997 Winter Simulation Conference*, Andradóttir, S. et al., eds. (Piscataway, NJ: IEEE).[6]
- Marsaglia, G. 1999, "Random Numbers for C: End, at Last?", posted 1999 January 20 to sci.stat.math.[7]
- Marsaglia, G. 2003, "Diehard Battery of Tests of Randomness v0.2 beta," 2007+ at <http://www.cs.hku.hk/~diehard/>. [8]
- Rukhin, A. et al. 2001, "A Statistical Test Suite for Random and Pseudorandom Number Generators", NIST Special Publication 800-22 (revised to May 15, 2001).[9]
- Marsaglia, G. 2003, "Xorshift RNGs", *Journal of Statistical Software*, vol. 8, no. 14, pp. 1-6.[10]
- Brent, R.P. 2004, "Note on Marsaglia's Xorshift Random Number Generators", *Journal of Statistical Software*, vol. 11, no. 5, pp. 1-5.[11]
- L'Ecuyer, P. 1996, "Maximally Equidistributed Combined Tausworthe Generators," *Mathematics of Computation*, vol. 65, pp. 203-213.[12]
- L'Ecuyer, P. 1999, "Tables of Maximally Equidistributed Combined LSFR Generators," *Mathematics of Computation*, vol. 68, pp. 261-269.[13]
- Couture, R. and L'Ecuyer, P. 1997, "Distribution Properties of Multiply-with-Carry Random Number Generators," *Mathematics of Computation*, vol. 66, pp. 591-607.[14]
- L'Ecuyer, P. 1999, "Tables of Linear Congruential Generators of Different Sizes and Good Lattice Structure", *Mathematics of Computation*, vol. 68, pp. 249-260.[15]

7.2 Completely Hashing a Large Array

We introduced the idea of a random hash or *hash function* in §7.1.4. Once in a while we might want a hash function that operates not on a single word, but on an entire array of length M . Being perfectionists, we want every single bit in the hashed output array to depend on every single bit in the given input array. One way to achieve this is to borrow structural concepts from algorithms as unrelated as the Data Encryption Standard (DES) and the Fast Fourier Transform (FFT)!

DES, like its progenitor cryptographic system LUCIFER, is a so-called "block product cipher" [1]. It acts on 64 bits of input by iteratively applying (16 times, in fact) a kind of highly nonlinear bit-mixing function. Figure 7.2.1 shows the flow of information in DES during this mixing. The function g , which takes 32 bits into 32 bits, is called the "cipher function." Meyer and Matyas [1] discuss the importance of the cipher function being nonlinear, as well as other design criteria.

DES constructs its cipher function g from an intricate set of bit permutations and table lookups acting on short sequences of consecutive bits. For our purposes, a different function g that can be rapidly computed in a high-level computer language is preferable. Such a function probably weakens the algorithm cryptographically. Our purposes are not, however, cryptographic: We want to find the fastest g , and the smallest number of iterations of the mixing procedure in Figure 7.2.1, such that our output random sequence passes the tests that are customarily applied to random number generators. The resulting algorithm is not DES, but rather a kind of "pseudo-DES," better suited to the purpose at hand.

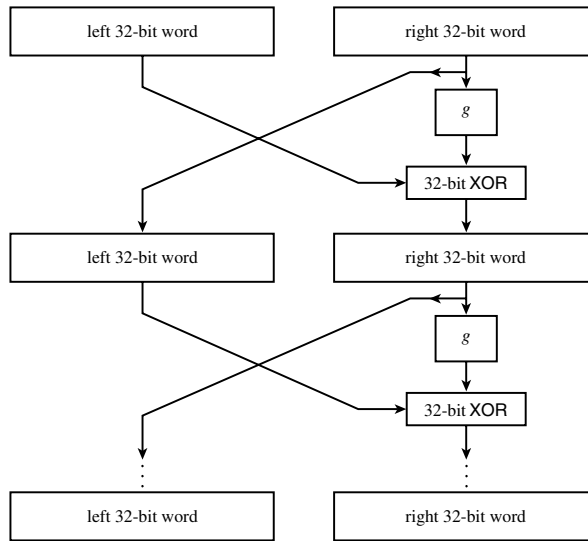


Figure 7.2.1. The Data Encryption Standard (DES) iterates a nonlinear function g on two 32-bit words, in the manner shown here (after Meyer and Matyas [1]).

Following the criterion mentioned above, that g should be nonlinear, we must give the integer multiply operation a prominent place in g . Confining ourselves to multiplying 16-bit operands into a 32-bit result, the general idea of g is to calculate the three distinct 32-bit products of the high and low 16-bit input half-words, and then to combine these, and perhaps additional fixed constants, by fast operations (e.g., add or exclusive-or) into a single 32-bit result.

There are only a limited number of ways of effecting this general scheme, allowing systematic exploration of the alternatives. Experimentation and tests of the randomness of the output lead to the sequence of operations shown in Figure 7.2.2. The few new elements in the figure need explanation: The values C_1 and C_2 are fixed constants, chosen randomly with the constraint that they have exactly 16 1-bits and 16 0-bits; combining these constants via exclusive-or ensures that the overall g has no bias toward 0- or 1-bits. The “reverse half-words” operation in Figure 7.2.2 turns out to be essential; otherwise, the very lowest and very highest bits are not properly mixed by the three multiplications.

It remains to specify the smallest number of iterations N_{it} that we can get away with. For purposes of this section, we recommend $N_{it} = 2$. We have not found any statistical deviation from randomness in sequences of up to 10^9 random deviates derived from this scheme. However, we include C_1 and C_2 constants for $N_{it} \leq 4$.

```

void psdes(Uint &lword, Uint &rword) {
    const int NITER=2;
    static const Uint c1[4]={
        0xbaa96887L, 0x1e17d32cL, 0x03bcd3c3L, 0x0f33d1b2L};
    static const Uint c2[4]={
        0x4b0f3b58L, 0xe874f0c3L, 0x6955c5a6L, 0x55a7ca46L};
    Uint i,ia,ib,iswap,itmph=0,itmpl=0;
    for (i=0;i<NITER;i++) {
        Perform niter iterations of DES logic, using a simpler (noncryptographic) nonlinear function
        instead of DES's.
        ia = (iswap=rword) ^ c1[i];
        itmpl = ia & 0xffff;
        itmph = ia >> 16;
    }
}

```

Pseudo-DES hashing of the 64-bit word (lword,rword). Both 32-bit arguments are returned hashed on all bits.

The bit-rich constants c1 and (below) c2 guarantee lots of nonlinear mixing.

hashall.h

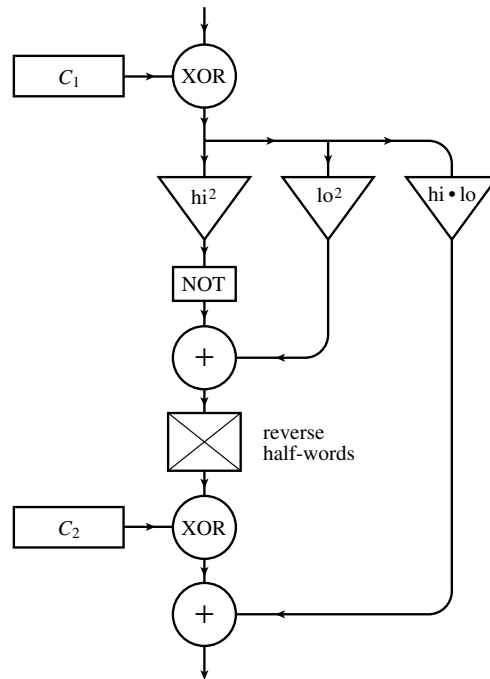


Figure 7.2.2. The nonlinear function g used by the routine `psdes`.

```

    ib=itmpl*itmpl+ ~(itmph*itmph);
    rword = lword ^ (((ia = (ib >> 16) |
        ((ib & 0xffff) << 16)) ^ c2[i])+itmpl*itmph);
    lword = iswap;
}
}

```

Thus far, this doesn't seem to have much to do with completely hashing a large array. However, `psdes` gives us a building block, a routine for mutually hashing two arbitrary 32-bit integers. We now turn to the FFT concept of the *butterfly* to extend the hash to a whole array.

The butterfly is a particular algorithmic construct that applies to an array of length N , a power of 2. It brings every element into mutual communication with every other element in about $N \log_2 N$ operations. A useful metaphor is to imagine that one array element has a disease that infects any other element with which it has contact. Then the butterfly has two properties of interest here: (i) After its $\log_2 N$ stages, everyone has the disease. Furthermore, (ii) after j stages, 2^j elements are infected; there is never an “eye of the needle” or “necking down” of the communication path.

The butterfly is very simple to describe: In the first stage, every element in the first half of the array mutually communicates with its corresponding element in the second half of the array. Now recursively do this same thing to each of the halves, and so on. We can see by induction that every element now has a communication path to every other one: Obviously it works when $N = 2$. And if it works for N , it must also work for $2N$, because the first step gives every element a communication path into both its own and the other half of the array, after which it has, by assumption, a path everywhere.

We need to modify the butterfly slightly, so that our array size M does not have to be a power of 2. Let N be the next larger power of 2. We do the butterfly on the (virtual) size N , ignoring any communication with nonexistent elements larger than M . This, by itself, doesn't do the job, because the later elements in the first $N/2$ were not able to “infect” the second $N/2$ (and similarly at later recursive levels). However, if we do one extra communication between

elements of the first $N/2$ and second $N/2$ at the very end, then all missing communication paths are restored by traveling through the first $N/2$ elements.

The third line in the following code is an idiom that sets n to the next larger power of 2 greater or equal to m , a miniature masterpiece due to S.E. Anderson [2]. If you look closely, you'll see that it is itself a sort of butterfly, but now on bits!

```
void hashall(VecUInt &arr) {
```

hashall.h

Replace the array `arr` by a same-sized hash, all of whose bits depend on all of the bits in `arr`. Uses `psdes` for the mutual hash of two 32-bit words.

```
    Int m=arr.size(), n=m-1;
    n|=n>>1; n|=n>>2; n|=n>>4; n|=n>>8; n|=n>>16; n++;
    Incredibly, n is now the next power of 2  $\geq m$ .
    Int nb=n,nb2=n>>1,j,jb;
    if (n<2) throw("size must be > 1");
    while (nb > 1) {
        for (jb=0;jb<n-nb+1;jb+=nb)
            for (j=0;j<nb2;j++)
                if (jb+j+nb2 < m) psdes(arr[jb+j],arr[jb+j+nb2]);
        nb = nb2;
        nb2 >>= 1;
    }
    nb2 = n>>1;
    if (m != n) for (j=nb2;j<m;j++) psdes(arr[j],arr[j-nb2]);
    Final mix needed only if m is not a power of 2.
```

```
}
```

CITED REFERENCES AND FURTHER READING:

- Meyer, C.H. and Matyas, S.M. 1982, *Cryptography: A New Dimension in Computer Data Security* (New York: Wiley).[1]
- Zonst, A.E. 2000, *Understanding the FFT*, 2nd revised ed. (Titusville, FL: Citrus Press).
- Anderson, S.E. 2005, "Bit Twiddling Hacks," 2007+ at <http://graphics.stanford.edu/~seander/bithacks.html>. [2]
- Data Encryption Standard*, 1977 January 15, Federal Information Processing Standards Publication, number 46 (Washington: U.S. Department of Commerce, National Bureau of Standards).
- Guidelines for Implementing and Using the NBS Data Encryption Standard*, 1981 April 1, Federal Information Processing Standards Publication, number 74 (Washington: U.S. Department of Commerce, National Bureau of Standards).

7.3 Deviates from Other Distributions

In §7.1 we learned how to generate random deviates with a uniform probability between 0 and 1, denoted $U(0, 1)$. The probability of generating a number between x and $x + dx$ is

$$p(x)dx = \begin{cases} dx & 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases} \quad (7.3.1)$$

and we write

$$x \sim U(0, 1) \quad (7.3.2)$$

As in §6.14, the symbol $\sim$ can be read as "is drawn from the distribution."

In this section, we learn how to generate random deviates drawn from other probability distributions, including all of those discussed in §6.14. Discussion of specific distributions is interleaved with the discussion of the general methods used.